

APUNTESWEB · INGENIERÍA INFORMÁTICA

COGA

Segundo curso · 2.º cuatrimestre · 9 temas

JULIO DE 2026 · [HTTPS://ADRIANQL5.GITHUB.IO/APUNTESWEB/](https://adrianql5.github.io/apuntesweb/)

1. Introducción

1.1 Introducción

El **ojo humano** percibe el **movimiento** de manera continua captando numerosas imágenes estáticas que el cerebro usa para dar sensación de fluidez.

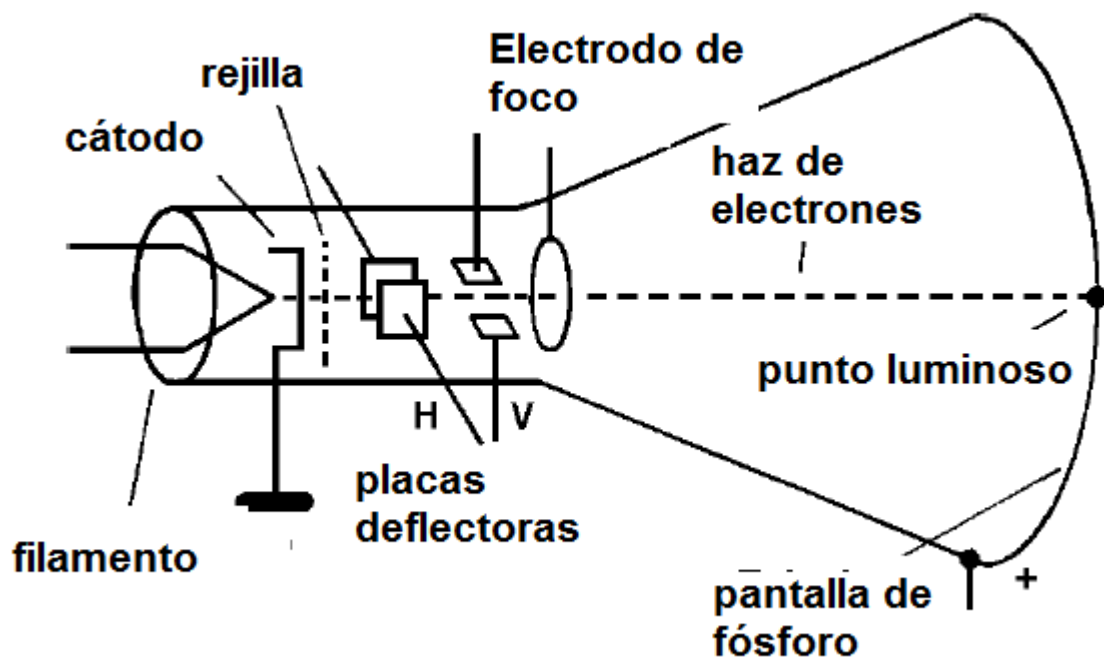
- **Persistencia:** fenómeno por el cual una imagen permanece durante un breve periodo después de que la fuente de luz haya desaparecido. Permite ver secuencias de imágenes como un movimiento continuo
- **Motion Blur (desenfoque de movimiento):** efecto de desenfoque que ocurre cuando un objeto en movimiento rápido deja una estela en la imagen. En el cine se usa para hacer el movimiento más realista.

Las imágenes en movimiento que vemos se forman con una serie de **imágenes fijas** mostradas en **rápida sucesión**.

- **Frecuencia de refresco:** cuántas veces por segundo se actualiza una imagen en una pantalla. Para simular la visión humana se necesitan $24 - 30\text{fps}$

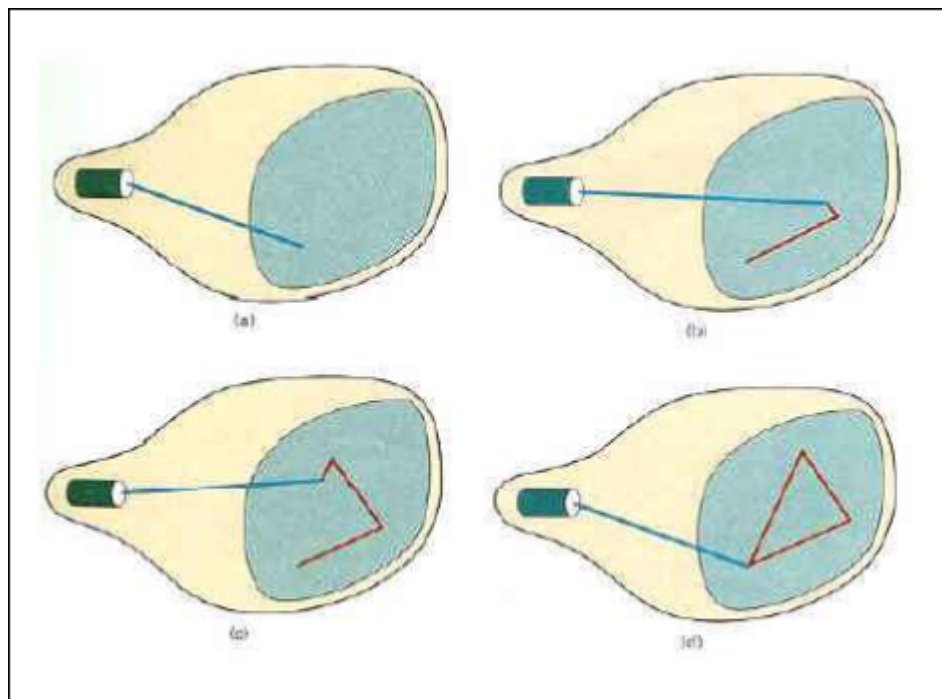
1.2 Hardware Gráfico (TRC)

El **tubo de rayos catódicos (TRC)** consiste en un haz de e^- (haz catódico) emitido por un cañón de e^- que pasa a través de sistemas de enfoque y reflexión que lo redirigen hacia posiciones específicas de una pantalla revestida de fósforo. El fósforo emite luz en la posición alcanzada por los e^- .

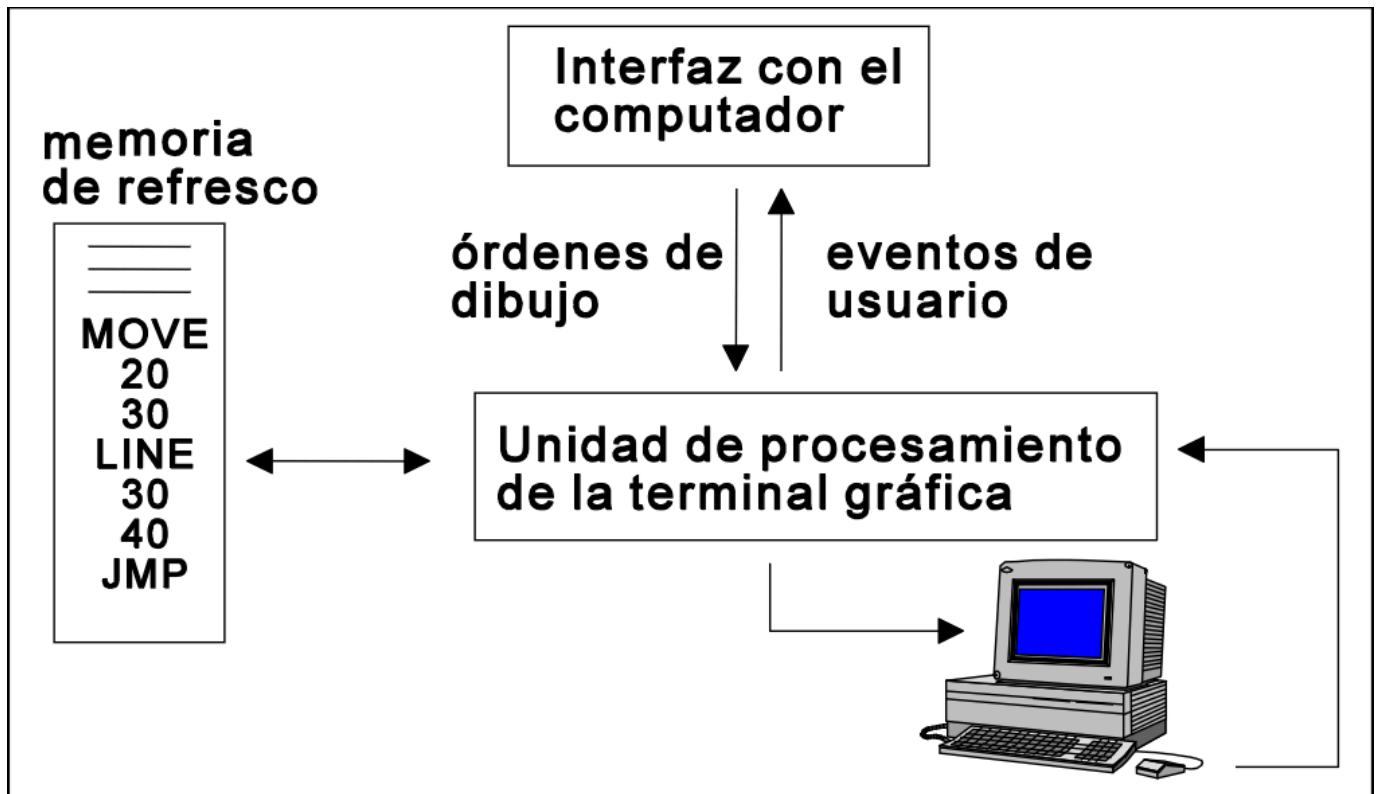


1.2.1 Terminales Vectoriales

Terminales vectoriales: TRC en el que el haz de e^- recorre aleatoriamente la pantalla de acuerdo con unas órdenes de dibujo.

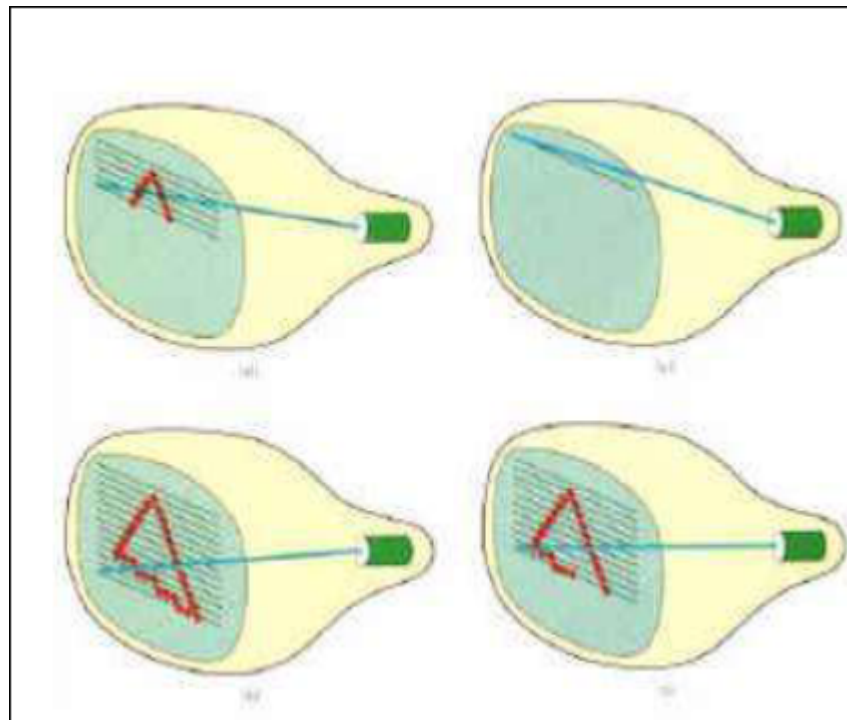


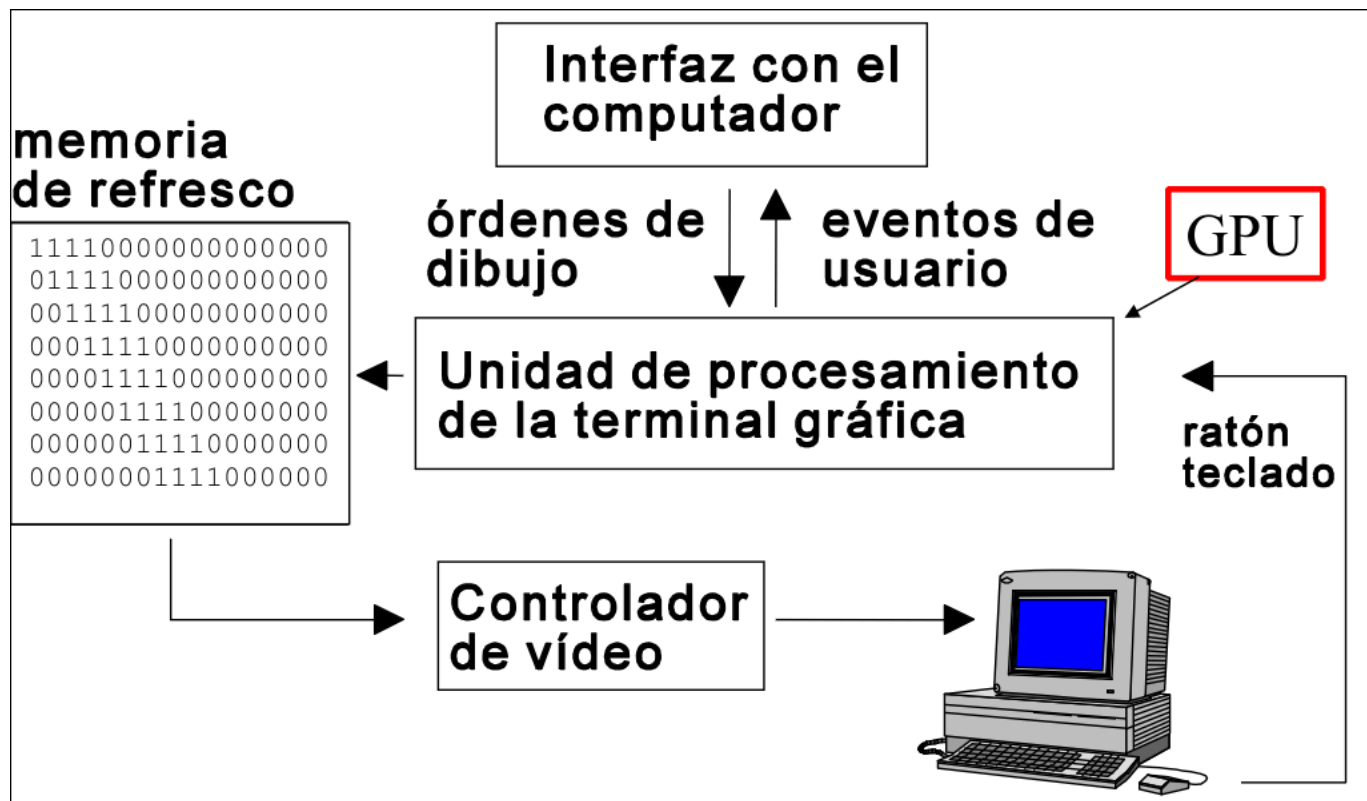
- **Unidad de Procesamiento de la Terminal Gráfica:** procesa las órdenes de dibujo y las traduce en comandos para la pantalla. Accede y actualiza la memoria de refresco para mantener la imagen visible
- **Memoria de refresco:** almacena las instrucciones necesarias para redibujar la imagen en la pantalla constantemente para que no desaparezca.



1.2.2 Terminales Raster

Terminales raster: TRC en el que el haz de e^- recorre la pantalla por líneas (625 líneas), desde la parte superior izquierda a la inferior derecha. Los datos mostrados por pantalla se extrapolan a una **matriz bidimensional** en la que cada entrada es un **Píxel**, intersección entre fila y columna.



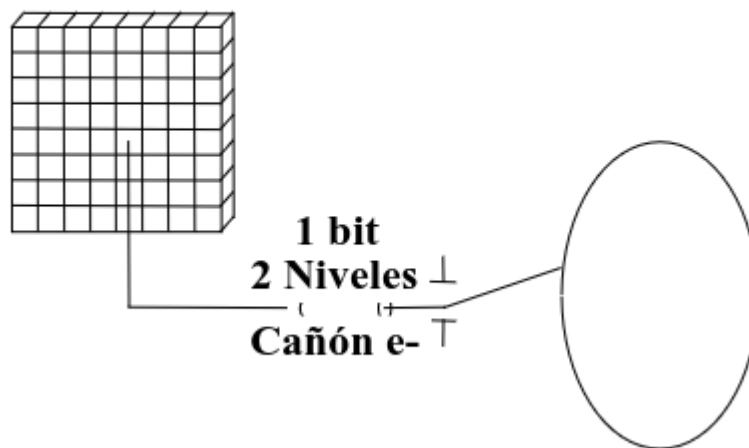


El conjunto de valores de los píxeles se almacena en una zona de memoria denominada **Frame-Buffer**. El dispositivo o aplicación varía sus valores y estos se vuelcan (**swap**) a la pantalla en un momento dado.

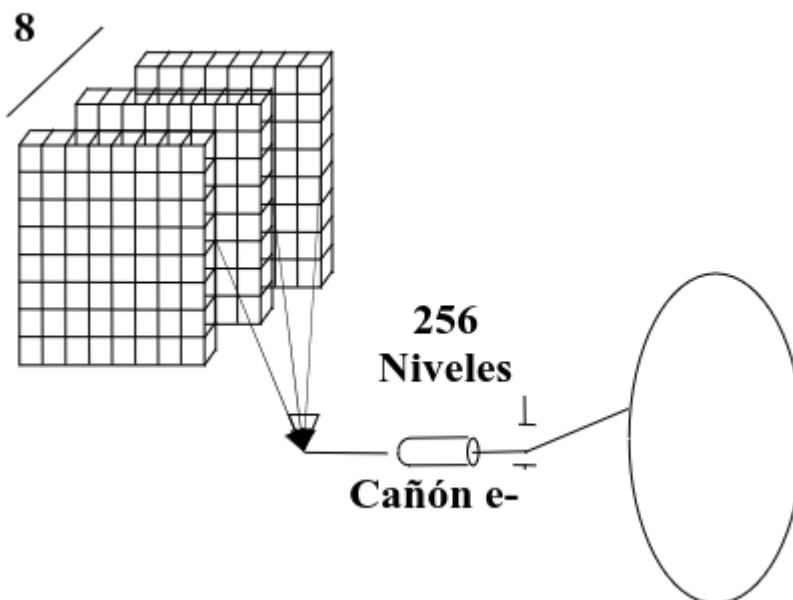
La **profundidad de color** (*bpp*) es el número de **bits por pixel**, y determina la cantidad de **colores** que se pueden mostrar por pantalla:

- $bpp \leq 8$ -> los valores de los píxeles son tonos RGB
- $bpp > 8$ -> se usa el modelo RGB estándar
- Para obtener el **color real** se necesitan 8 bits para cada color (RGB) y otros 8 (A) de transparencia, en total 32 (RGBA).
- Para **ahorrar memoria**, se puede usar una pelta **LUT** con una entrada para cada posible valor de un píxel en una imagen, permitiendo indexar los colores usando el mínimo número de bits posible.

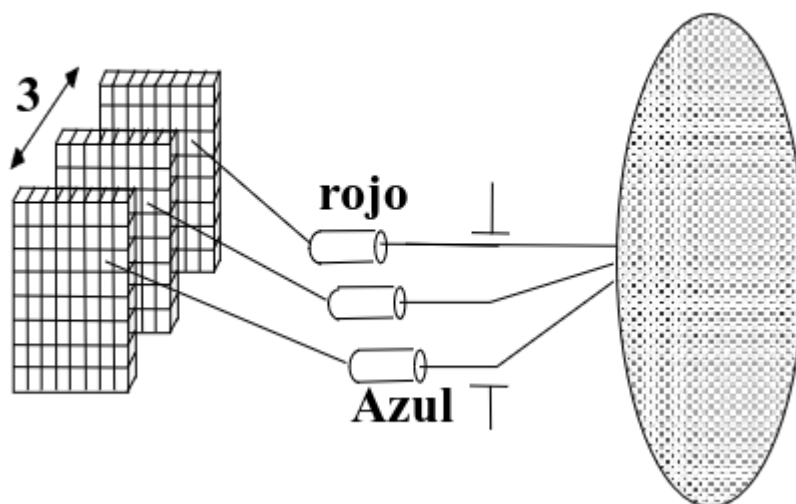
Pantalla monocromo



256 Niveles de gris



16 Colores



Otros tipos de terminales ráster con distintas **bpp, frecuencia de refresco y resolución** son pantallas de plasma, LC, LED, OLED, etc.

Las 625 líneas se abandonaron rápidamente ya que estas estaban limitadas debido a la capacidad de emisión. Al poder procesar más píxeles y aumentar la velocidad de refresco se disminuye el cansancio ocular del usuario.

1.3 Conceptos Básicos

- **Ráster:** representación de imágenes como una matriz de píxeles.
- **Pixel (picture element):** cada uno de los puntos de color que se muestran en una pantalla
- **Gráficos vectoriales:** formato de imagen compuesto por objetos geométricos (puntos, líneas, polígonos), cuyos atributos matemáticos se almacenan. Permiten modificar el tamaño de una imagen sin pérdidas de calidad.
- **Renderizado:** proceso de creación de una imagen a partir de un modelo, calculando el color de cada píxel en la imagen según las propiedades del modelo.

1.3.1 Double buffer

Se usan dos **frame buffer**:

- Un **buffer primario** cuyo contenido se muestra por pantalla.
- Un **buffer secundario** o trasero en el que se realiza el renderizado de las imágenes. Cuando se termina de renderizar en el buffer secundario, se copia su contenido al principal. Así se evita que el espectador pueda ver el **cambio entre frames** si no se renderiza lo suficientemente rápido.

- Se dibuja en el buffer secundario - Una vez lista la imagen, se realiza un ****swap****, mostrando instantáneamente el nuevo frame en el buffer primario. - Se limpia el buffer secundario y se repite.

1.3.2 Depth Buffer

El **Z-Buffer** o buffer de profundidad se usa para gestionar la **profundidad** de los objetos y resolver cuáles son **visibles y cuáles están detrás de otros**. Tiene el mismo ancho y alto que el frame-buffer y una profundidad de 16, 24, o 32 bits.

Cada píxel tiene un valor de profundidad (Z) que **indica su distancia con respecto a la cámara**, de manera que cuando se renderiza un nuevo píxel se compara su valor con el del anterior. En función del resultado, lo reemplazará o será descartado.

El algoritmo del pintor consiste en dibujar los objetos de **atrás hacia adelante**, ordenándolos según su coordenada Z (de mayor a menor). Si dos objetos intersecan, no sabe determina cuál debe dibujarse primero.

1.4 Modelo de Cámara Sintética

Elementos de una escena 3d: - Geometría - Texturizado - Iluminación - Cámara

1.5 Pipeline

Pipeline gráfica: modelo conceptual que describe los pasos que un sistema gráfico debe dar para renderizar una escena 3D en una pantalla 2D.

- **Vertex Shader:**
 - **Recibe la posición del vértice del triángulo** + otros datos (coordenadas de textura, normal, color, etc.)
 - **Devuelve el vértice con su posición transformada** + (opcionalmente) otros atributos (normal, color, etc), que se envían al siguiente paso en la pipeline.
- **Fragment Shader:**
 - Recibe información del fragmento a procesar, que puede incluir datos interpolados desde los vértices (coordenadas de textura, normal, color, etc.)
 - Devuelve el color final del píxel si este se debe representar en pantalla.
- Procesos posteriores:
 - **Z-BUFFER:** controla la profundidad de los píxeles para manejar la superposición de objetos en la escena
 - **Alpha Test:** determina si un objeto debe renderizarse en función de su transparencia
 - **Blending Test**

2. Modelado

2.1 Objetos Sólidos

Modelar un objeto es darle forma. Existen dos tipos de modelado:

- **Modelado de interior:** se suelen emplear representaciones basadas en Spatial-Partitioning que consiste en describir el interior de un objeto diviéndolo en regiones pequeñas sólidas que no se solapan entre sí. Por lo general se usan cubos (marching cubes)
- **Modelado de Exterior o Representación de Superficie:** define el contorno del objeto sin hacer referencia a su volumen interior. Proporciona información sobre:
 - **Superficie:** debe tener en cuenta su geometría
 - **Propiedades visuales:** cómo se comporta el color o la textura frente a la luz
 - Alguna **propiedad física** (como la elasticidad). Si va a efectuar una simulación física sobre el objeto.

En la representación de objetos 3D las superficies representadas deben ser cerradas.

2.2 Ejes, Puntos y Vectores

Tanto los puntos, como los vectores y los objetos, se representarán sobre los tres ejes cartesianos.

- x , anchura
- y , altura
- z , profundidad

Por defecto, el rango de valores de las coordenadas va desde $(-1,-1,-1)$ a $(1,1,1)$.

2.3 Triangulación

Para modelar cualquier objeto se emplea la técnica de **teselación**, que consiste en su representación mediante polígonos. Por lo general, se suele usar el triángulo, por lo que se llama **triangulación**.

$$\text{Número de triángulos} = \text{Número de lados del polígono} - 2$$

2.4 Identificación de Caras

Un plano se puede definir por su **vector normal**, que se puede calcular mediante el producto vectorial de dos de sus vectores. Si simplemente se desea conocer la **dirección** del vector normal, basta aplicar la **regla de la mano derecha** según el recorrido de los vértices. Si se recorren en sentido antihorario la normal será positiva, en caso contrario, será negativa

CALCULAR VECTORES ENTRE 2 PUNTOS

$$A(x_1, y_1, z_1), B(x_2, y_2, z_2) \quad \vec{AB} = (x_2 - x_1, y_2 - y_1, z_2 - z_1)$$

El producto vectorial $\vec{u} \times \vec{v}$ en R^3 se calcula como el determinante de la matriz 3×3 conformada por los vectores unitarios de los ejes, los componentes del vector $\vec{u}$ y los del $\vec{v}$:

$$\begin{vmatrix} \vec{i} & \vec{j} & \vec{k} \\ \vec{u}_1 & \vec{u}_2 & \vec{u}_3 \\ \vec{v}_1 & \vec{v}_2 & \vec{v}_3 \end{vmatrix}$$

Dados los vectores:

$$\mathbf{u} = (1, 0, 0), \quad \mathbf{v} = (0, 1, 0)$$

Calculamos el producto cruz:

$$\mathbf{u} \times \mathbf{v} = \begin{vmatrix} \mathbf{i} & \mathbf{j} & \mathbf{k} \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{vmatrix}$$

La normal se puede especificar para un conjunto de vértices con `glNormal3f(x, y, z)` o `glNormal3fv(*v)`.

En OpenGL se usa `FACE CULLING`, es decir **sólo se visualizan las superficies cuya normal se dirige hacia la cámara**. Por tanto hay que tener en cuenta el **orden de creación de los vértices**. Está desactivado por defecto, pero esto se puede cambiar con `glEnable(GL_CULL_FACE)` y `glDisable(GL_CULL_FACE)`. Por defecto, se visualizarán las caras frontales (lo que se puede especificar con `glFrontFace(GL_CCW)` pero se puede desactivar con `glFrontFace(GL_CW)`).

Un vector normal se puede **normalizar** (es decir, transformar en un vector unitario) dividiendo cada una de sus componentes por su módulo. Esto es útil para procesos de **iluminación de objetos**, por lo que conviene activar la normalización automática con `glEnable(GL_NORMALIZE)`.

$$\vec{v}_{\text{normalizado}} = \frac{\vec{v}}{\|\vec{v}\|}$$
$$\|\vec{v}\| = \sqrt{v_1^2 + v_2^2 + \dots + v_n^2}$$

2.5 Tablas de Representación

Para la representación de un objeto en base a polígonos necesitamos conocer, representar y almacenar sus vértices y atributos en **tablas**. Tipos de tablas:

- **Geométricas:** almacenan información sobre los vértices y parámetros sobre los polígonos o superficie que determinan.
- **De atributos:** almacenan información sobre las propiedades físicas de los polígonos (textura, color, transparencia, etc.)

Tipos de tablas según sus índices:

- **Sin indexar**
- **Indexada:** hay una tabla para los vértices (con 3 coordenadas para cada uno) y otro para los polígonos (con los vértices que definen cada uno). Si un vértice se usa varias veces, se escribe una única vez en la tabla de vértices y se puede referenciarse varias veces en la de polígonos.
- **Doblemente indexada:** hay una tabla para los vértices (con 3 coordenadas para cada uno), otra para las aristas (con dos vértices para cada una y si se desea los dos polígonos que la comparte) y otra para los polígonos (con las aristas de cada uno).

3. Transformaciones

3.1 Transformaciones

En la **pipeline gráfica** se describen varios espacios de trabajo:

- El **Local Space** es el sistema de **coordenadas locales** de los objetos con relación a su origen local.
- El **World Space** es el sistema de **coordenadas de escena** o de mundo de los objetos con relación al origen general.
- La **Model Matrix** transforma las coordenadas locales de la escena, trasladando, rotando y escalando los objetos hasta colocarlos en su posición final.

Las **transformaciones geométricas** se usan para trasladar, escalar y rotar los **objetos** y la **cámara de la escena**.

En la **pipeline fija (Opengl 1.2)** el paso de coordenadas locales a coordenadas de la escena se corresponde con el paso **transformación geométrica**.

En el **retained mode (Opengl 3.3)** el paso de coordenadas locales a coordenadas de escena se realiza mediante el **Vertex Shader**.

3.1.1 Traslación

Llamamos **traslación** a mover un objeto (todos sus vértices) de un lugar a otro. Si queremos trasladar un punto $P(x, y)$ a una nueva posición $P'(x', y')$, usamos la ecuación:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} T_x \\ T_y \end{bmatrix} = \begin{bmatrix} x + T_x \\ y + T_y \end{bmatrix}$$
$$P' = T + P$$

En Opengl 1.2 `glTranslatef(Tx, Ty, Tz)`

3.1.2 Rotación respecto al Origen

Llamemos **rotación respecto al origen** a desplazar un ángulo θ un objeto (todos sus vértices) respecto al origen. Para rotar un punto $P(x, y)$ usamos relaciones trigonométricas:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix}$$

$$P' = R * P$$

En OpenGL 1.2 `glRotatef( $\theta$ , x, y, z)` donde θ está en grados y el giro se realiza alrededor del eje que pasa por (0,0,0) y (x,y,z) en sentido contrario a las agujas del reloj.

3.1.3 Rotación General

Para desplazar un ángulo θ un objeto (todos sus vértices) respecto un punto cualquiera (x_c, y_c) :

- Traslación al origen.
- Rotación respecto al origen
- Traslación a (x_c, y_c)

$$P' = T(x_c, y_c) * R(\alpha) * T(-x_c, -y_c) * P$$

3.1.4 Ángulos de Euler

Dados dos sistemas de coordenadas xyz e XYZ con un mismo origen, es posible determinar la posición de uno en términos del otro usando tres ángulos $\alpha \beta \gamma$. Esto sirve para representar los ejes de un objeto respecto a un sistema de referencia general. Los giros son **acumulativos** y los **ángulos** se encuentran **ya girados**. Para usar ángulos de Euler en `glRotatef( $\theta$ , x, y, z)` se pondría (1, 0, 0), (0, 1, 0), (0, 0, 1).

3.1.5 Escalado respecto al origen

Llamamos **escalado** a cambiar el tamaño de un objeto (todos sus vértices). No es una transformación de **cuerpo rígido** ya que la geometría varía. Para escalar multiplicamos por los **coeficientes de escalado**, que serán la razón entre las coordenadas nuevas y viejas:

$$S_x = \frac{\text{anchura nueva}}{\text{anchura vieja}}$$

$$S_y = \frac{\text{altura nueva}}{\text{altura vieja}}$$

- Si $S_x = S_y$ se realiza un **escalado uniforme**
- $S_x \neq S_y$ se realiza un **escalado no uniforme**

Para escalar un punto $P(x, y)$ multiplicamos sus coordenadas por los coeficientes de escalado:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} S_x & 0 \\ 0 & S_y \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix}$$

$$P' = S * P$$

En OpenGL(1.2) `glScale(Sx, Sy, Sz)`

3.1.6 Escalado General

Para escalar un objeto (todos sus vértices) cuando el **origen de coordenadas no está en su interior** sin producir un desplazamiento:

- Traslación al origen
- Escalado respecto al origen
- Traslación al punto inicial

3.2 Coordenadas Homogéneas

Para poder realizar todas las transformaciones mediante **multiplicación de matrices** hay que cambiar la forma en la que se realiza la traslación. Para esto utilizaremos las **coordenadas homogéneas**, que añaden un nuevo elemento w que será la última columna de las matrices de transformación, la cual se usará para colocar los factores de traslación.

- En los puntos $w = 1$
- En los vectores $w = 0$
- No existe ni el (0,0,0) ni el (0,0,0,1).

3.2.1 Transformaciones en 2D

$$T(x, y) = T(x, y, 1)$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} T_x \\ T_y \end{bmatrix} = \begin{bmatrix} x + T_x \\ y + T_y \end{bmatrix} \Rightarrow \begin{pmatrix} 1 & 0 & T_x \\ 0 & 1 & T_y \\ 0 & 0 & 1 \end{pmatrix}$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix} \Rightarrow \begin{pmatrix} \cos \alpha & -\sin \alpha & 0 \\ \sin \alpha & \cos \alpha & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} S_x & 0 \\ 0 & S_y \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix} \Rightarrow \begin{pmatrix} S_x & 0 & 0 \\ 0 & S_y & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

3.2.2 Transformaciones en 3D

Traslación

$$T(d_x, d_y, d_z) = \begin{bmatrix} 1 & 0 & 0 & d_x \\ 0 & 1 & 0 & d_y \\ 0 & 0 & 1 & d_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Escalado

$$S(s_x, s_y, s_z) = \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Rotación alrededor del eje Z

$$R_z(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta & 0 & 0 \\ \sin \theta & \cos \theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Rotación alrededor del eje X

$$R_x(\theta) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta & 0 \\ 0 & \sin \theta & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Rotación alrededor del eje Y

$$R_y(\theta) = \begin{bmatrix} \cos \theta & 0 & \sin \theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \theta & 0 & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Notación en columna $M[16]$ Por ejemplo, los valores de traslación estarán en $M(13, 14, 15, 16)$.

3.2.3 Zonas de una matriz

La zona azul serían **transformaciones lineales**, la verde **traslaciones** y la rosa **ceros**.

$$\begin{bmatrix} N_{11} & N_{21} & N_{31} & N_{41} \\ N_{12} & N_{22} & N_{32} & N_{42} \\ N_{13} & N_{23} & N_{33} & N_{43} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

4. Visión

4.1 View Space

El **View Space** es el sistema de **coordenadas de cámara** o de punto de vista del observador. La **View Matrix** transforma las coordenadas de escena a coordenadas de cámara mediante una combinación de traslaciones y rotaciones.

No existe una matriz separada para aplicar transformaciones a la cámara:

- En el view space, la **cámara** se ubica siempre en el $(0, 0, 0)$ mirando hacia el eje $-Z$
- Para conseguir esto, se deberían realizar ciertas transformaciones sobre la posición de la cámara en el world space.
- En su lugar, a la **escena** se le aplica la **inversa** de las **transformaciones** que se le aplicarían a la **cámara**.

El **Clip Space** es el sistema de coordenadas tras el **recorte** de los elementos que están fuera del frustrum. Las coordenadas que quedan en el clip space son **procesadas** y se pueden convertir en **fragmentos** visibles por pantalla.

El **Frustrum** es el volumen de visión de la cámara, es decir, el área del espacio que la cámara verá. Por defecto, el frustrum es $[-1, 1], [-1, 1]$. Este rango corresponde al **espacio de coordenadas normalizadas**.

La **Projection Matrix** permite cambiar las dimensiones del frustrum. Después de aplicar la matriz de proyección se realiza una **división perspectiva** que convierte las **coordenadas del clip space a NCD (normalized device coordinates)**, dividiendo cada componente x, y, z por el componente homogéneo w .

En la **pipeline fija (OpenGL 1.2)** el paso de coordenadas de escena a coordenadas del clip space se corresponde con el paso **transformación geométrica**.

En el **retained mode (OpenGL 3.3)** el paso de coordenadas de escena a coordenadas del clip space se realiza mediante el **vertex shader**.

4.2 Proyección

Una **proyección** es una transformación que reduce la dimensionalidad de un espacio, normalmente de R^3 a R^2 . Es decir, es el proceso de convertir una **escena tridimensional** en una **imagen bidimensional**

4.2.1 Elementos de una proyección

- **Escena:** elementos 3D que componen el mundo virtual
- **Plano de proyección:** superficie 2D en la que se representan los objetos 3D. En la práctica se corresponde a la pantalla o viewport.
- **Punto de vista o centro de proyección:** posición desde donde se observa la escena. En la práctica se corresponde a la ubicación de la cámara
- **Proyectores:** rayos imaginarios que se trazan desde los puntos del objeto hasta el CP. Los puntos en los que intersecan con el PP determinan dónde se dibujará cada punto del objeto en la imagen final.

4.2.2 Tipos de Proyecciones

Proyección ortográfica o paralela

El CP está situado en el infinito, por lo que los proyectores son paralelos entre sí. Se define una **dirección de proyección** en vez de un CP

Proyección perspectiva

El CP está situado en el espacio, por lo que los proyectores forman un cono cuyo vértice es el CP.

Proporciona **realismo visual**, da **sensación tridimensional**, no mantiene la forma ni la escala.

Proyección ortográfica multivista.

El PP es perpendicular a algún eje, por lo que se necesitan **varias vistas** para obtener una visión 3D de los objetos.

- **Planta:** el PP es perpendicular al eje Y
- **Alzado:** el PP es perpendicular al eje Z
- - **Perfil:** el PP es perpendicular al eje X Es posible realizar **medidas precisas**. Todas las vistas tienen la **misma escala**. No se obtiene una **visión 3D** del objeto.

5. Iluminación

Una vez realizado el ensamblado de vértices, se pasa al proceso de **rasterizado**, en el que se transforman los objetos que se van a proyectar en una matriz de **píxeles** que se mostrarán por pantalla. Así se obtiene la **correspondencia** de los **puntos de los objetos** con los **puntos de la pantalla**. Esta información compone un **fragmento**.

En la **pipeline fija (OpenGL 1.2)** hay poco control sobre el proceso de rasterizado e interpolación .

En el **retained mode (OpenGL 3.3)** el **fragment shader** permite trabajar a muy bajo nivel sobre los fragmentos.

5.1 La Luz

La luz es una onda electromagnética que es percibida por el ser humano con un **color** determinado por su **frecuencia** o una suma de frecuencias. El conjunto de las frecuencias o longitudes de onda se llama **espectro**.

5.1.1 Caracterización Humana de la Luz

En el ojo humano hay dos tipos fundamentales de **detectores**:

- **Bastones:** células fotorreceptoras especializadas en la visión nocturna que responden a la **luminosidad**, por lo que **no distinguen colores**. Tienen su máxima sensibilidad en la luz verde.
- **Conos:** células fotorreceptoras responsables de la visión diurna y la **percepción del color**. Hay 3 tipos, cada uno sensible a distintos colores:
 - **Rojos o conos-L:** longitudes de onda largas
 - **Verdes o Conos-M:** longitudes de onda medias.
 - **Azules o Conos-S:** longitudes de onda cortas. Son más sensibles a la luz que las demás

El **espectro** de la intensidad luminosa, en función de la longitud de onda λ es :

$$I(\lambda) = I_R(\lambda) + I_V(\lambda) + I_A(\lambda)$$

Por esta razón, en las imágenes de síntesis **se caracteriza un color** por medio de sus **componentes RGB**. Existen **modelos más precisos** que realizan una discretización mucho más fina del espectro luminoso.

5.1.2 Teoría del Color: Colores Aditivos

La **teoría de los colores aditivos** se basa en la combinación de luz para crear diferentes colores. En esta teoría los tres **colores primarios** son rojo, verde y azul. La suma de estos colores permite crear todos los demás.

- La **mezcla de dos** colores primarios en proporciones iguales produce **colores secundarios**
- La mezcla de los **tres** colores primarios en intensidades máximas produce **luz blanca**.
- La **ausencia total** produce negro

En OpenGL los colores de los vértices se especifican con `glColor3f(R,G,B)`, con valores para los componentes RGB en un rango [0,1] o [0,255]. OpenGL usa la síntesis aditiva porque los **monitores y pantallas** funcionan emitiendo luz en estos tres colores primarios.

5.1.3 Interpolación del Color

En OpenGL, se interpola el **color de las caras** a partir del color de los **vértices** que las forman. Por ejemplo, para obtener la siguiente imagen, se dibujan 3 triángulos y la intensidad final es la suma de los tres.

5.2 Modelos de Iluminación

Hay dos grandes tipos de modelos de iluminación:

- **Locales:** sólo consideran las características inmediatas del punto y su entorno cercano, sin tener en cuenta las interacciones complejas de la luz con otros objetos de la escena. Se utilizan en aplicaciones donde prima la **velocidad de renderizado**. El más clásico es el modelo de **Phong**.
- **Globales:** consideran la escena en conjunto y las interacciones complejas de la luz en toda la escena, incluyendo los efectos indirectos entre objetos. Se utilizan en aplicaciones donde prima la **calidad**, porque aunque son más lentos, los resultados son mejores.

5.2.1 Modelo Local de Iluminación

Los cálculos se realizan en función de:

- Fuentes de luz, caracterizadas por:
 - **Posición**
 - **Intensidad y color**
 - **Número de luces**
 - **Distribución luminica:** uniforme, focalizada, direccional, etc.
 - **Geometría:** puntual, esférica, lineal, etc.

- **Objetos**, caracterizados por:
 - **Distancias**: al observador y a la fuente de luz.
 - **Material / Propiedades ópticas**: transparencia, refracción, etc.
 - **Cromaticidad**: color superficial propio
 - **Interacción lumínica** con otros objetos de la escena
- **Observador**, caracterizado por:
 - **Dirección de observación**: cálculo de la intensidad

5.2.2 Modelo de Iluminación Simple

Resumiendo lo explicado anteriormente, la iluminación en un punto de una escena puede representarse mediante la siguiente función:

$$I = f(p, p_v, \{MGMO\}, \{MGCL\})$$

Donde:

- I : Intensidad luminosa observada en el punto pp.
- p : Punto donde se calcula la iluminación.
- p_v : Posición del observador.
- $\{MGMO\}$: Modelo geométrico y material de los objetos.
- $\{MGCL\}$: Modelo geométrico y cromático de la luz.

Para simplificar el modelo, se pueden asumir ciertas condiciones:

- Las **fuentes de luz** son puntuales (pequeñas y redondas).
- Los **objetos son opacos** y no emiten luz propia.
- No existen **inter-reflexiones**: la luz no rebota de un objeto a otro.
- Los objetos no generan sombras entre sí.

5.2.3 Modelo de Iluminación de Phong

El **modelo de phong** es un **modelo empírico de iluminación local** caracterizado por usar **fuentes de luz puntuales** y por que los **objetos** (no fuentes de luz) **no emiten luz**.

Luz Ambiental

Es una luz **no direccional no identificable**. Su efecto solo depende de su color y del color del objeto. Su variación implica cambios uniformes en la iluminación de los objetos.

No genera sombras ni matices de color. Indice sobre **todas las partes** del objeto por igual Muestra los objetos como **siluetas**.

$$I = I_a \times K_a$$

Donde:

- I_a : Intensidad ambiental en todo punto del espacio.
- K_a : Coeficiente de reflexión ambiental del objeto (**entre 0 y 1**). Un valor de 1 significa que el objeto refleja toda la luz ambiental.

Luz Difusa o Luz Lambert Está asociada a un ****foco de luz****. Su variación implica cambios de intensidad o de posición del foco. Genera sombras y matices de color en función del ángulo con el que incide en la superficie.

Proviene de una **dirección concreta** y es **reflejada** en todas **direcciones** según la ley de Lambert, afectando aquellas **partes** del objeto en las que **incide**.

Ley de Lambert: "La componente difusa de la luz reflejada por una superficie es proporcional al coseno del ángulo de incidencia con la normal de la superficie"

$$I = I_l \times \cos(\theta) = I_l \times K_D \times (\vec{N} \cdot \vec{L})$$

Donde

- I_L : Intensidad de la fuente de luz.
- K_D : Coeficiente de reflexión difusa del material (**entre 0 y 1**).
- θ : Ángulo de incidencia entre la normal y la dirección de la luz (**entre 0° y 90°**).
- $\vec{N}$: Vector normal a la superficie (unitario).
- $\vec{L}$: Vector de iluminación (unitario).

Luz Especular o Luz de Phong

Está asociada a un **foco intenso de luz**. Se usa en objetos con brillo, en extremos en un espejo. Su variación está asociada a cambios de intensidad, de posición del foco y de punto de vista.

Proviene de **una dirección concreta** y depende de la **posición del observador** y de las **propiedades del objeto**. Está asociada a objetos **brillantes o pulidos**: en el caso ideal, con la luz reflejada exactamente en el ángulo incidente, produce un espejo.

- Superficies perfectas: el rayo incidente y el reflejado son coplanares
- Superficies imperfectas: la luz se refleja dentro de un patrón tridimensional de reflexión. Hay luz fuera del plano y el ángulo ideal de reflexión. Las **reflexiones** especulares suele ser del mismo **color** que la **fuentes de luz**.

Se calcula con la ecuación:

$$I = I_L \times K_S \times \cos^n(\alpha) = I_L \times K_S \times (\vec{R} \cdot \vec{V})^n$$

Donde:

- I_L : Intensidad de la fuente de luz.
- K_s : Coeficiente de reflexión especular (**entre 0 y 1**). Depende del material del objeto
- n : Exponente de brillo (**1 color mate e inf espejo**).
- $\vec{R}$: Vector de reflexión de la luz.
- $\vec{V}$: Vector que apunta al observador.

Luz Emisiva

Da apariencia de que el objeto emite luz.

5.3 Modelo Global de Iluminación Simple

$$I = I_a \times K_a + I_L \times (K_S \times (\vec{R} \cdot \vec{V})^n + K_d \times (\vec{N} \cdot \vec{L}))$$

Otros **efectos**:

- **Atenuación**: no desestimar la atenuación de la luz con distancia al observador
- **Múltiples fuentes**: para tener varias fuentes en cuenta sólo hay que sumar sus efectos.
- **Color**: para tener en cuenta los tres colores hay que triplicar las expresiones:

$$\begin{aligned} I_R &= I_{aR} \cdot K_a \cdot C_{dR} + I_{LR} \cdot f_{at} \cdot \left(K_d \cdot C_{dR} \cdot (\vec{N} \cdot \vec{L}) + K_S \cdot C_{eR} \cdot (\vec{R} \cdot \vec{V})^n \right) \\ I_G &= I_{aG} \cdot K_a \cdot C_{dG} + I_{LG} \cdot f_{at} \cdot \left(K_d \cdot C_{dG} \cdot (\vec{N} \cdot \vec{L}) + K_S \cdot C_{eG} \cdot (\vec{R} \cdot \vec{V})^n \right) \\ I_B &= I_{aB} \cdot K_a \cdot C_{dB} + I_{LB} \cdot f_{at} \cdot \left(K_d \cdot C_{dB} \cdot (\vec{N} \cdot \vec{L}) + K_S \cdot C_{eB} \cdot (\vec{R} \cdot \vec{V})^n \right) \end{aligned}$$

5.4 Resumen

6. Texturas

Una vez realizado el ensamblado de vértices se pasa al proceso de **rasterizado**, en el que se transforman los **objetos** que se van a proyectar en una matriz de **píxeles** que se mostrarán por pantalla. Así se obtiene la **correspondencia** de los **puntos de los objetos** con los **puntos de la pantalla**. Esta información compone un **fragmento**.

En la **pipelin fija (OPengl 1.2)** hay poco control sobre el proceso de rasterizado e interpolación.

En el **retained mode (Opengl 3.3)** el **fragment shader** permite trabajar a muy bajo nivel sobre los fragmentos.

6.1 Introducción

Una **textura** es una colección o array n-dimensional de valores que se representan sobre la geometría de los objetos.

- Aumentan el **realismo de la imagen** sin incrementar el **número de polígonos**.
- Proporcionan una clave para la percepción de la **profundidad**.
- Permiten simular muchos **efectos**: sombras, entornos, etc.
- Ocupan Memoria

Tipos de Textura:

- **1D**: imágenes sin alto, solo ancho
- **2D**: imagenes bitmap ordinarias
- **3D**: apilamientos de texturas. Ocupan mucha memoria
- **Texturas procedurales**: la imagen es calculada a partir de un algoritmo matemático.
 - Ocupan poca memoria, pero consumen CPU
 - Se pueden **reescalar** con facilidad, por lo que son útiles para superficies de cualquier tamaño.
 - Ejemplos de mapas generados: geométricos, patrón repetitivo, ruido fractal, etc.

6.2 Texturizado

El **texturizado** consiste en superponer sobre los polígonos de un objeto una textura.

6.2.1 Texture Mapping

Crea una textura a partir de una **imagen** para aplicarla sobre la superficie, como si fuera una pegatina.

6.2.1 Bump Mapping

Perturba la superficie modificando las **normales** de las caras del objeto, pero **sin cambiar su geometría**. En sentido estricto puede no considerarse texturizado, es un tipo de **mapeado topológico**

- Las normales se calculan en función de un **mapa de alturas** (una matriz de datos $2^n * 2^n$, generalmente una imagen).
- Se combinan las normales con la geometría del objeto
- Se calcula el modelo de iluminación para esos nuevos valores de las normales.

6.2.3 Environment Mapping

Mapea el **entorno** del objeto sobre su superficie.

6.2.4 Light Mapping

El brillo de las superficies se calcula previamente y se almacena para aplicarse como **mapa de iluminación** sobre la superficie. Se suelen usar en objetos estáticos para proporcionar efectos de iluminación como los de la global con un coste muy bajo.

6.2.5 Displacing Mapping

Modifica la geometría de un objeto mediante un mapa de texturas, normalmente cambiando la dirección de las **normales**.

- Genera una gran cantidad de **geometría adicional**
- Es muy **lento**.
- Se pueden generar **sombras**
- Se pueden generar **oclusiones**
- Se puede usar para **sombreados**
- Geometry shader-> shader que se aplica después del **vertex shader** para generar geometría

6.3 Mapeado de Texturas

El **mapeado de texturas** es la correspondencia de los puntos de la textura con los puntos de la superficie en la que se aplica. Tipos de mapeado:

- Mapeado **directo**: la textura se mapea directamente sobre el objeto
- Mapeado **en dos fases**: la textura se mapea sobre un objeto 3D y se proyecta sobre el objeto

6.3.1 Mapeado Directo de Texturas

Una textura 2D se presenta como un patrón de datos 2D $T(u, v)$, donde el par (u, v) se denominan **coordenadas de Textura**. El **mapa de texturas** asocia un punto $T(u, v)$ con un punto (x, y, z) de la geometría del objeto. En general el mapa de texturas es muy **complejo**, por lo que se suele usar un programa de diseño CAD.

6.3.2 Mapeado de Texturas en OpenGL

- Habilitar el uso de texturas 1D, 2D o 3D.
- Activar la textura
- Almacenar en memoria, dando valores o leyendo un fichero
- Definir la textura
- Mapear la textura

Habilitar el Mapeado de Texturas

En el `main` o en el `InitOpenGL`: `glEnable(GL_TEXTURE_2D)`: Si se quiere deshabilitar para que alguna superficie aparezca sin textura, con el color original: `glDisable(GL_TEXTURE_2D)`

Leer y Activar la Textura

- `void glGenTextures(GLsizei n, GLuint*textura)`
 - `n`: número de índices a crear
 - `textura`: puntero al primer elemento del array donde se almacenan los índices creados.
- `void glBindTexture(GLenum target, GLuint texture)`
 - `target`: `GL_TEXTURE_1D` o `GL_TEXTURE_2D`
 - `texture`: índice de la textura que se quiere enlazar (si se quiere no usar textura, 0).
- `glDeleteTextures(GLsizei num, const GLuint *nombres)`

Almacenar en Memoria

Una textura se puede especificar como un array de **texels**, que se puede crear **manualmente**.

Normalmente, o bien se escribe un programa que la **genere** (texturas procedurales) o se lee de un **fichero de imagen** con la librería STB:

Definir la Textura

```
void glTexImage2D(objetivo, nivel, componentes, ancho, alto, borde, formato, tipo, *píxeles)
```

- `objetivo`: indica cuál es la textura a definir (`GL_TEXTURE_1D` o `GL_TEXTURE_2D`)
- `nivel`: indica el nivel de detalle de las texturas, normalmente 0.
- `componentes`: indica el número de componentes de color usados en cada pixel.
- `ancho`: indica el ancho de la textura, que debe ser potencia de 2
- `alto`: indica el alto de la textura, que debe ser potencia de 2
- `borde`: indica el número de píxeles de borde que se usarán.
- `formato`: indica el tipo de valores esperados (normalmente `GL_RGB`)
- `tipo`: indica el tipo de la variable que almacena los valores de color por pixel.
- `píxeles`: puntero a la variable que almacena los colores de cada pixel

Pegar la Textura - OpenGL 1.2

Con dos texturas:

Pegar la Textura - Opengl 3.3

La carga es igual, pero hay que especificar las coordenadas de textura.

6.3.3 Mapeado Multicapa en Opengl 3.3

Se usan **unidades de textura** que permiten ir activando o desactivando las texturas manualmente

6.4 Parámetros de Texturas

Sobre el **objeto**: problemas de **tamaño**: qué hacer si la textura no abarca todo el objeto.

- `glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S/T, GL_REPEAT/CLAMP)`: especifica qué ocurre si la textura no cubre los límites del objeto, puede repetirse (`GL_REPEAT`), o estirarse (`GL_CLAMP`). Debe llamarse dos veces, para la vertical (*T*) y para la horizontal (*S*)

Sobre la **pantalla**: problemas de **escala**: qué hacer si al rasterizar, cada texel de la textura no se corresponde exactamente con un pixel de la pantalla.

- `glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN/MAG_FILTER, GL_LINEAR/NEAREST)`. Si la superficie es mayor que la textura, cada pixel se corresponderá con un trozo pequeño de texel: magnificación. Si la superficie es menor que la textura, cada pixel se corresponderá con un conjunto de texels contiguos: minificación.

Mezclas: que hacer si el objeto tenía un color.

- `glTexEnvf(GL_TEXTURE_ENV, GL_TEXTURE_ENV_MODE, GLint valor)`: para establecer el modo RGBA donde `valor` puede ser (`p`=pixel, `s`=textura, `c`=entorno):
 - `GL_REPLACE`: $C_V = C_T$ reemplaza el color del objeto por la textura.
 - `GL_MODULATE`: $C_V = C_S * C_T$ se multiplica el color del objeto y el de la textura.
 - `GL_DECAL`: $C_V = C_T * C_\alpha$ sustituye teniendo en cuenta el canal Alpha
 - `GL_BLEND`: $C_V = C_P * (1 - C_S) + C_C * C_S$ multiplica el color del píxel por el color de la textura y del entorno un color elegido.
- `glTexEnvfv(GL_TEXTURE, GL_TEXTURE_ENV_COLOR, GLfloat color[4])` para enviar el color.

6.5 Transparencia

Blending: la forma más sencilla de hacer una transparencia es mediante la técnica de puerta o **ventana transparente**. Se usa una ecuación de la forma $c = a_s * c_s + (1 - a_s) * c_t$ siendo c_t el color objetivo y c_s e color fuente.

El modelo RGBA acepta el valor alpha. Para ellos las texturas o color deben tener un alpha, por ejemplo .png o .gifa. Se habilita con `glEnable(GL_ALPHA_TEST)`.

El **alpha test** compara el valor del fragmento con un valor de referencia. Por defecto, este valor es el 0 y la comparación se hace entre 0 y 1.

- `glAlphaFunc(GLenum func, GLclamp ref)` donde `func` puede ser `GL_NEVER`, `GL_LESS`, `GL_EQUAL`, `GL_LEQUAL`, `GL_GREATER`, `GL_NOTEQUAL` o `GL_ALWAYS` (por defecto).

Para cualquier otro elemento de color RGBA:

- `glEnable(GL_BLEND)`
- `glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA)`

Nosotros trabajaremos con texturas que tengan ya incorporado canal Alpha, por lo que tendremos en cuenta el número de canales, por lo que hay que incluir este número en la función de lectura.

7. Otras Técnicas

7.1 Buffer de Profundidad

El **algoritmo del pintor** consiste en dibujar **primero** los objetos más **lejanos** de la escena según su coordenada z u después cubrirlos con los más cercanos.

- Si dos objetos **intersecan**, no sabe determinas cual dibujar primero
- Se renderizan **todos los puntos** de los polígonos, incluso si estos estarían ocultos en la escena final.

El **Z-Buffer** almacena, para cada píxel de la pantalla, la **profundidad(z)** del **objeto más cercano a la cámara en ese píxel**. Tiene el mismo **ancho y alto que el frame-buffer** y una **profundidad** de 16,24 o 32 bits (a mayor profundidad mayor precisión).

La **prueba de profundidad** consiste en que, cuando se va a renderizar un objeto sobre un píxel en el que ya había otro, se compara su valor de profundidad (z) con le que ya hay almacenado en el z-buffer.

- Si el nuevo está más **cerca** de la cámara (menor z), se dibujará y sustituirá al antiguo en el z-buffer.
- Si el nuevo está mas **lejos** de la cámara (mayor z) se descartará.

En principio, la prueba de profundidad se realiza después del **fragment shader**. Hoy en día la mayoría de GPUs admiten una **prueba de profundidad temprana**, realizando el test **antes del fragment shader** para incrementar la **performance**. Consiste en que, siempre esté claro que un fragmento **no será visible** (pues está detrás de otros) se **descartará prematuramente**.

El **z-buffer** almacena la profundidad de un rango normalizado entre 0 (más cercano) y 1 (más lejano) para que se **compare rápidamente**, mientras que en el **view space** la coordenada z está comprendida entre el **near y el far** del frustrum.

En realidad, la **precisión** con la que se almacena la profundidad de los objetos **no es uniforme** en ese rango, los objetos lejanos se comprimen mas y los cercanos tienen más detalle, por lo que la relación entre la coordenada z real y el valor almacenado en el buffer (z') sigue una función no lineal.

$$z' = \frac{1}{z}$$
$$z' = \frac{\frac{1}{z} - \frac{1}{\text{near}}}{\frac{1}{\text{far}} - \frac{1}{\text{near}}}$$

`glEnable(GL_DEPTH_TEST)` : activa la prueba de profundidad `glDisable(GL_DEPTH_TEST)` : desactiva la prueba de profundidad, por lo que dibuja según el orden en el que recibe los fragmentos, aunque

estén detrás `glClear(GL_DEPTH_BUFFER_BIT)`: limpia el z-buffer `glDepthFunc(func)`: modifica los operadores de comparación usados en la prueba de profundidad.

Por defecto, se usa `GL_ALWAYS`, que implementa el **algoritmo del pintor**, pero se pueden usar:

- `GL_NEVER`, `GL_LESS`, `GL_EQUAL`, `GL_LEQUAL`, `GL_GREATER`, `GL_NOTEQUAL`, o `GL_ALWAYS`.

7.1.1 Z-buffer 3.3

En opengl 3.3 se puede visualizar el z-buffer accediendo a `gl_FragCoord.z` en el **fragment shader**, por lo que se puede transformar fácilmente su valor 0-1.

```
void main(){
    FragColor = vec4(vec3(gl_FragCoord.z), 1.0);
}
```

Como es de esperar, los valores obtenidos no son lineales, pero se pueden linealizar usando la ecuación antes planteada:

```
#version 330 core
out vec4 FragColor;
float near = 0.1;
float far = 10.0;

float no_lineal(float depth){
    float z = depth*2.0-1.0;
    return (2.0*near*far)/(far+near.z*(far-near))
}

void main(){
    float depth = lineal (gl_FragCoord.z)/far;
    FragColor = vec4(vec3(depth),1.0);
}
```

7.1.2 Z-Fighting

El **z-fighting** es un **artefacto visual** en el que dos objetos tienen una profundidad tan similar que el z-buffer **no tiene suficiente precisión** para determinar cual está delante, por lo que ambas parecen cambiar continuamente de orden, causando efectos extraños. Como la precisión de la profundidad en zonas cercas a la cámara es mayor, es más común en **objetos lejanos**

Posibles **soluciones**:

- Aumentar la precisión del z-buffer si es posible
- Elegir bien el near y far
- Separar un poco los objetos

7.2 Atenuación

Atenuación: la intensidad de iluminación decrece a medida que aumenta la distancia a la fuente de luz.

$$Atenuación = \frac{1}{a + bd + cd^2}$$

donde d es distancia entre la fuente y el punto que se está iluminando.

OpenGL establece por defecto $a = 1$ y $b = c = 0$, pero estos son valores que se pueden modificar con: `void glLightf(GL_LIGHTX, GL_CONSTANT/LINEAR/CUADRATIC_ATTENUATION, valor)`

Solo influye sobre las componentes **ambiente y difusa** de las luces. Además, puede afectar negativamente al **rendimiento**.

7.3 Niebla

La **niebla** es un efecto especial utilizado para **difuminar** los objetos según su distancia, haciendo que los más lejanos se mezclen con otro color y parezcan desvanecerse.

El efecto niebla se puede aplicar de dos formas:

- Basado en **tablas** (cálculo por píxel), más preciso
- Basado en **vértices**, más eficiente pero menos preciso.

`glEnable(GL_FOG)`, activa la niebla.

Para especificar la densidad, color y tipo de niebla: `glFogfv(parametro, valor)` donde `parametro` puede ser:

- `GL_FOG_DENSITY`, `GL_FOG_START`, `GL_FOG_END`, `GL_FOG_INDEX`, `GL_FOG_COLOR`

`glFogi(GL_FOG_MODE, GL_LINEAR/EXP/EXP2)`

- `GL_LINEAR` -> $f = \frac{end-z}{end-start}$
- `GL_EXP` -> $f = e^{-density*z}$
- `GL_EXP2` -> $f = e^{-(density*z)^2}$

7.4 MipMapping

El **mipmapping** consiste en asociar **varias texturas** de **distintas calidades** a la misma superficie para mostrar una de ellas en función de su profundidad.

Para usarlo, se debe modificar la función de lectura de la textura.

```
glTexImage2D(GL_TEXTURE_2D, 0, GL_RGB, width, height, 0, GL_RGB, GL_UNSIGNED_BYTE,
data);

gluBuild2DMipmaps(GL_TEXTURE_2D, GL_RGB, ancho, alto, GL_RGBA, GL_UNSIGNED_BYTE,
ptr);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR_MIPMAP_LINEAR);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);

// gluBuild2DMipmaps – builds a two-dimensional mipmap
GLint gluBuild2DMipmaps(GLenum target, GLint internalFormat, GLsizei width, GLsizei
height, GLenum format, GLenum type, const void * data);

unsigned char *data = stbi_load(path, &width, &height, &nrComponents, 0);
if (data)
{
    gluBuild2DMipmaps(GL_TEXTURE_2D, GL_RGB, width, height, GL_RGB, GL_UNSIGNED_BYTE,
data); // con mipmap
    glGenerateMipmap(GL_TEXTURE_2D);
}
```

7.5 SkyBox

La **skybox** es una caja que contiene el escenario en su interior sobre cuyo interior se aplica una textura de cielo. También se puede usar una semicircunferencia o *dome*.

7.6 Colisiones

Teóricamente, para saber si dos objetos están colisionando se debe comprobar **para cada una de las caras** de ambos si se interseca con alguna de las del otro. Como esto es muy costoso, se suelen **envolver los objetos** en una **forma geométrica** más simple, como una **caja**, una **esfera**, un cilindro o un elipsoide.

7.6.1 Bounding Spheres

Las **boundings spheres** están definidas por un **centro** (x,y,z) y un **radio**(float). Dos esferas colisionan si la **distancia entre sus centros es menor que la suma de sus radios**.

$$dist = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2 + (z_1 - z_2)^2}$$

```
if(dist<(r1+r2)->colision)
```

7.6.2 Bounding Boxes Las **bounding boxes** están definidas por un **punto mínimo** (xMin, yMin, zMin) y un **punto máximo** (xMax, yMax, zMax)

Hay dos manera de comprobar si las dos cajas colisionan: - **Axis Aligned Bounding Boxes (AAB)**: se usan **cajas alineadas con los ejes**. Se basa en comprobar si **alguna de las 8 esquinas** de la caja está dentro de la caja de otro: ```c++ if(AxMin < BxMax && AxMax > BxMin && AyMin < ByMax && AyMax > ByMin && AzMin < BzMax && AzMax > BzMax) → colisión ``` El método es más fácil, rápido y barato de calcular.

- **Separated Axis Test (SAT)**: se usan **segmentos, triángulos y cajas no alineadas con los ejes**.
 - Dos objetos **no colisionan si existe al menos un eje de proyección en el cual al proyectarlo no se superponen**.

7.7 Árboles, particionado del espacio y mapas

Programar en 1.2

1 Crear una Ventana

```
#include <windows.h>
#include <glut.h>
#include <gl.h>
#include <glu.h>

const int W_WIDTH = 500;    // Ancho de la ventana
const int W_HEIGHT = 500;   // Alto de la ventana

void myDibujo() {
    glClear(GL_COLOR_BUFFER_BIT|GL_DEPTH_BUFFER_BIT); //borramos ambos buffers

    glBegin(GL_POINTS); //Indicamos que vamos a dibujar puntos
    glColor3f(0.0, 1.0, 0.0); //Indicamos el color del punto
    glVertex3d(0.5, 0.5,0.5);

    glFlush(); //limpia la pantalla (forzamos la impresion del dibujo por
pantalla mejor dicho)
    glSwapBuffers();//al usar double buffer debemos forzar el intercambio
}

void iniciar() {
    glClearColor(1, 0, 0, 1); //le decimos que limpie la pantalla de rojo
    glClearDepth(1.0); //especificamos el valor de limpieza del z-buffer
}

int main(int argc, char** argv) {

    glutInit(&argc, argv);//con esto iniciamos glut

    glutInitDisplayMode(GLUT_RGBA | GLUT_DOUBLE);//Indicamos que vamos a usar
RGBA y un buffer doble

    glutInitWindowSize(W_WIDTH, W_HEIGHT); //indicamos las dimensiones de
nuestra ventana
```

```

        glutInitWindowPosition(100, 100); //indicamos la posicion de la ventana
        glutCreateWindow("FUCK USC"); //creamos nuestra ventana y le ponemos un
        titulo descriptivo

        iniciar();

        glutDisplayFunc(myDibujo); //le indicamos que funcion debe usar para dibujar
        en pantalla

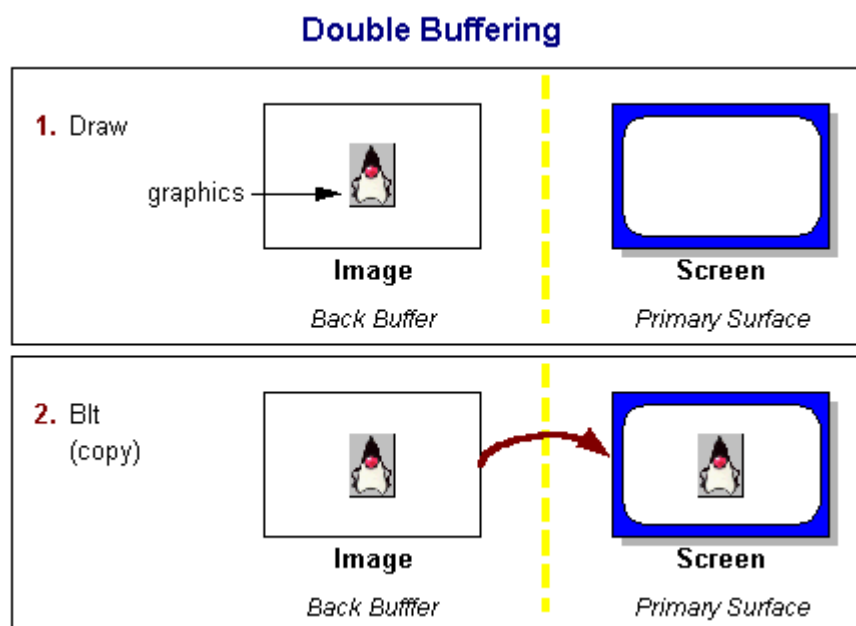
        glutMainLoop(); //esto es un bucle infinito que crea glut para que se
        muestre todo el rato la imagen en pantalla

        return 0;
    }

```

Comentarios:

- Usamos un buffer doble-> implica que se va dibujando en un buffer la imagen y después se intercambia con el otro buffer.



- RGBA-> red green blue y a es para la intensidad
- Usamos el z-buffer-> que almacena información sobre que objetos están más alejados de la cámara, le damos como valor por defecto 1.0 al borrado de este buffer, si ponemos 0.0 no se verá casi ningún objeto.

2 Modelado

2.1 Primitivas

Una **primitiva** es la interpretación de un conjunto ordenado de vértices que se conectan de manera distinta en función del tipo de primitiva.

Para especificar un vértice se usa `glVertex`, que puede tomar de 2 a 4 parámetros de cualquier tipo numérico (2D con $z = 0$ si se especifican 2 parámetros, 3D si se especifican 3 y coordenadas homogéneas si se especifican 4).

Para delimitar los vértices se usa `glBegin(MACRO)` y `glEnd()`.

>[Nota] >Si se introduce un número incorrecto de vértices, se ignorarán los últimos sobrantes

Por defecto, un punto se dibuja como un píxel en la pantalla y una línea como un segmento de un píxel de grosor.

- El tamaño de los puntos se puede cambiar con `glPointSize(tamaño)` y el grosor de las líneas con `glLineWidth(grosor)`.
- El color de los puntos se puede cambiar con `glColor3f(R,G,B)`

```
#include <windows.h>
#include <glut.h>
#include <gl.h>
#include <glu.h>

void myDibujo() {
    glClear(GL_COLOR_BUFFER_BIT|GL_DEPTH_BUFFER_BIT);

    glBegin(GL_LINES); //Indicamos que vamos a dibujar puntos
    glColor3f(1.0f, 0.0f, 0.0f); //Indicamos el color del punto
    glVertex3f(0.0f, 0.0f,0.0f);
    glVertex3f(0.0f, 5.0f,0.0f);
    glEnd(); //Le decimos que hemos acabado

    //No es necesario poner todo el rato glEnd y GLbegin si vais a usar un mismo
    tipo de figura, pero es util para entender vuestro código

    glFlush();
    glutSwapBuffers();
}
```

```
}
```

2.2 Listas de visualización

Una **lista de visualización** es un índice que nos permite renderizar un conjunto de vértices previamente almacenados simplemente invocándola.

- `glGenLists(rango)`: obtiene identificadores válidos para listas de visualización, donde `rango` es el número de índices contiguos deseados
- `glNewList(lista, modo)`: define una nueva lista de visualización con el identificador `lista` y `modo` `GL_COMPILE` o `GL_COMPILE_AND_EXECUTE`.
- `glEndList()`: finaliza la definición de la lista de visualización.
- `glCallList(lista)`: invoca la lista de visualización con identificador `lista`.

```
#include <GL/glut.h>

GLuint miLista; //declaro la lista

void crearDisplayList() {
    miLista = glGenLists(1); // Generar un id
    glNewList(miLista, GL_COMPILE); // Inicia la lista en modo COMPILE -> se guarda
    pero no se ejecuta
    glColor3f(1.0, 0.0, 0.0); // Color rojo
    glBegin(GL_TRIANGLES); // Dibujar un triángulo
        glVertex2f(-0.5, -0.5);
        glVertex2f(0.5, -0.5);
        glVertex2f(0.0, 0.5);
    glEnd();
    glEndList(); // Finalizar la lista
}

void display() {
    glClear(GL_COLOR_BUFFER_BIT);
    glCallList(miLista); // Llamar a la lista para dibujar el triángulo
    glFlush();
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
    glutInitWindowSize(500, 500);
    glutCreateWindow("Display Lists en OpenGL");
```

```

    crearDisplayList(); // Crear la Display List

    glutDisplayFunc(display);
    glutMainLoop();

    return 0;
}

```

2.3 Formas Básicas de GLUT

- Cono-> `glutSolidCone()`
- Toro-> `glutSolidTorus()`
- Dodecaedro-> `glutSolidDodecahedron()`
- Octaedro-> `glutSolidOctahedron()`
- Icosaedro-> `glutSolidIcosahedron()`
- Tetera-> `glutSolidTeapot()`.

También existe la versión `Wire` en vez de `Solid`

3 Transformaciones

`glMatrixMode(GL_MODELVIEW)`: para especificar que se desea trabajar con la matriz de transformación de modelo. `glLoadIdentity(void)`: reemplaza la matriz actual por la identidad 4x4

Hay dos posibles métodos para colocar los valores apropiados en la matriz de transformación:

- Como la matriz se almacena como un **array de 16 elementos**, estos se pueden establecer **manualmente**. `glLoadMatrix(array)` para carga el `array` en la matriz activa.

- La otra alternativa a estas funciones es usar las funciones de **transformación** (`glTranslatef`, `glRotatef`, `glScalef`). Las matrices T , R y S que generan estas funciones se van multiplicando sobre la matriz activa. El producto de matrices **no es conmutativo**, así que hay que tener cuidado con el orden en el que se usan estas funciones: **la primera función en aplicarse será la última en escribirse en el código**. - Los movimientos que depende unos de otros se pueden heredar consecutivamente. Para ello, se almacenan versiones de la matriz de transformación en una **pila**, de manera que **la actual será la que está más arriba**. - `glPushMatrix()`: almacena en la pila una copia de la matriz activa en la segunda posición de la pila - `glPopMatrix()`: elimina de la pila la matriz que está en la primera posición.

4 Visión

En OpenGL 1.2 el trabajo sobre la cámara (la view matrix y projection matrix) se reúne en la **matriz de proyección** `GL_PROJECTION`.

- `glMatrixMode(GL_PROJECTION)` : para especificar que se desea trabajar con la matriz de proyección.
- `glLoadIdentity(void)` : reemplaza la matriz actual por la identidad 4x4
- `gluLookAt(eye_x, eye_y, eye_z, at_x, at_y, at_z, up_x, up_y, up_z)` : para determinar **dónde y cómo está dispuesta la cámara**.
 - `eye` indica el punto dónde está la cámara (por defecto (0,0,0))
 - `at` indica el punto hacia el que mira la cámara (por defecto (0,0,-1))
 - `up` indica el vector de orientación de la cámara (por defecto (0,1,0))
- `glOrtho3D(left, right, top, down, near, far)` : para realizar una **proyección ortográfica**
 - Los parámetros delimitan el frustum, que tiene forma de cubo (por defecto (-1,1,-1,1,-1,1)).
 - Una vez definido el frustum, se pueden generar las diferentes proyecciones multivista moviendo la cámara.
- `gluPerspective(fovy, aspect, near, far)` : para realizar una **proyección perspectiva**
 - `fovy` es el ángulo de de apertura del frustum, normalmente 45 o 60
 - `aspect` es la razón de aspecto (alto/ancho del PP), normalmente 1:1, 4:2, 16:9...
 - `near` determina la distancia desde la cámara hasta el PP.
 - `far` determina la distancia desde la cámara hasta el fondo del frustum

5 Iluminación (crazy shi)

Para iluminar una escena en OpenGL hacen falta varios **procesos**:

- Iluminar la escena
- Indicar cómo es el **modelo de iluminación**
- Indicar las **propiedades de los materiales**.

Las habilitaciones necesarias para llevar a cabo estos procesos se realizan mediante la función `glEnable` (normalmente antes del `glutMainLoop`):

- `glEnable(GL_DEPTH_TEST)` : habilita el **buffer de profundidad**, aumenta la velocidad.
- `glEnable(GL_CULL_FACE)` : habilita la **ocultación de caras traseras**, aumenta la velocidad.
- `glEnable(GL_LIGHTING)` : habilita los **cálculos de iluminación**.
- `glEnable(GL_LIGHTX)` : habilita una **luz concreta** (la número X).

- `glEnable(GL_COLOR_MATERIAL)` : habilita el **seguimiento del color**.
 - `glColorMaterial(face, Mode)` : indica la cara sobre la que se hace el seguimiento y para qué luces se lleva a cabo. Cada objeto reflejará estas luces según la definición previa de su color.
- `glShadeModel(GL_FLAT/GL_SMOOTH)` : habilitar el **modelo de sombreado**.

5.1 Tipos de Luces

Luces Básicas

Modelo de iluminación general (sin poner luces) que define una **luz ambiente básica** y especifica cómo se realiza la iluminación de la escena en cuanto a si las luces afectan a las dos **caras** de un objeto.

Propiedades (por defecto):

- `glLightModelfv(GL_LIGHT_MODEL_AMBIENT, ambiente)` , valores de la componente ambiente base
- `glLightModelfv(GL_LIGHT_MODEL_TWO_SIDE, TRUE)` , indica que las luces afectan a ambas caras de un material.
- `glLightModelfv(GL_LIGHT_MODEL_LOCAL_VIEWER, TRUE)` , coloca el modo de cálculo con el observador local, si no, todos los cálculos se hacen como si el observador estuviese situado sobre el eje Z.

Luces Direccionales

Están situadas en el **infinito** y **no tienen atenuación**. La dirección por defecto es (0,0,1,0).

```
float direccion[4]={1,1,1,0}
glLightfv(GL_LIGHT0, GL_POSITION, direccion)
```

Luces Locales

Se definen como las direccionales, pero el cuarto elemento del vector vale 1.

```
float posicion[4] = {1,1,1,1}
float direccion[4] = {1,1,1,1}
glLightfv(GL_LIGHT_0, GL_POSITION, posicion)
glLightfv(GL_LIGHT_0, GL_DIRECTION, direccion)
```

Focos de luz

Se definen como las locales pero con una determinada **apertura**. `glLightf(GL_LIGHT_0, GL_SPOT_CUTOFF, angulo)` , angulo entre 0 y 180.

5.2 Propiedades de la Luz

Para especificar las propiedades de la luz:

- `glLightf(Luz, Parámetro, Valor)` (escalar)
- `glLightfv(Luz, Parámetro, PunteroAVector)` (vector)

5.6 Propiedades de los Materiales

El modelo de iluminación de OpenGL aproxima el **color** de un material en función de la luz que **indice** sobre él y la que es capaz de **reflejar**.

Las propiedades sobre los **colores**, indican cómo influye cada una de las propiedades de un rayo de luz sobre el objeto.

- `GL_AMBIENT`: comportamiento respecto al componente **ambiente** de la luz
- `GL_DIFFUSE`: Comportamiento respecto al componente **difuso** de la luz.
 - Ambas determinan el color del material y suelen tener valores aproximados sino iguales
- `GL_SPECULAR`: comportamiento respecto al componente **especular** de la luz.
 - Se suele definir blanca o del color de la luz, de manera que los reflejos del material se degradan desde el color base de la luz incidente hasta el color definido en esta propiedad.

Las propiedades sobre la capacidad de **reflexión**:

- `GL_SHININESS`: concentración de los puntos que van a **reflejar** la luz. Puede ser uno sólo o toda la superficie del objeto. Cuanto mayor, mas metálico ($[0,128]$) por defecto 0
- `GL_EMISSION`: capacidad de simular la **emisión** de luz sin ser propiamente una fuente. Se especifican los colores de la luz que emite. Se usa para simular estrellas, lámparas o simulares.

Para especificar las propiedades del material en cada polígono:

- `void glMaterial{if}{v}(Cara, parámetro, punteroAVector)`

5.7 Cálculo del color en cada Vértice

5.8 Sombreado Plano

Cada cara tiene una **única normal** y cada uno de los puntos que la forman tiene el **mismo color**. La normal de cada cara se puede **calcular** a partir de los vértices o se puede **especificar** para un conjunto de ellos. Es el modelo de iluminación más **simple y eficiente**. `glShadeModel(GL_FLAT)`

5.9 Sombreados de Interpolación

Gourard Shading

Cada vértice tiene su **propia normal** y el color de cada punto de las caras se calcula mediante **interpolación bilineal de los colores de de sus vértices**. La normal de cada vértice se calcula por defecto como la **media de las normales de las caras** (calculadas a partir de los vértices que las forman) de las que forma parte ese vértice. También se puede **especificar para cada vértice**. Da un efecto de sombreado suave más **realista** para superficies **curvas**. `glEnable(GL_SMOOTH)`

```
glShadeModel(GL_SMOOTH)
```

Phong Shading

Cada vértice tiene su **propio normal** y la **normal de cada punto de las caras** se calcula mediante **interpolación bilineal**. Es la más **realista** de todas, pero consume más **recursos**. OpenGL **no la implementa** directamente, es **responsabilidad del programador** interpolar las normales y asociarlas a todos los puntos

Programar en 3.3

1 Crear una ventana

```
#include <glad/glad.h>
#include <GLFW/glfw3.h>
#include <iostream>

// Dimensiones de la ventana
const int SCR_WIDTH = 800;
const int SCR_HEIGHT = 600;

// Función para manejar entradas del teclado
void processInput(GLFWwindow* window) {
    if (glfwGetKey(window, GLFW_KEY_ESCAPE) == GLFW_PRESS)
        glfwSetWindowShouldClose(window, true);
}

int main() {
    // Inicializa GLFW
    if (!glfwInit()) {
        std::cerr << "Error al inicializar GLFW" << std::endl;
        return -1;
    }

    // Especificamos la versión de OpenGL (3.3 Core Profile)
    //lo de mayor es el 3 y lo de menor el .3
    glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 3);
    glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 3);
    //esto hace que elimine características obsoletas
    glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE);

    // Creamos la ventana, poniendo tamaño, y título.
    GLFWwindow* window = glfwCreateWindow(SCR_WIDTH, SCR_HEIGHT, "Ventana con GLFW",
    NULL, NULL);
    if (!window) {
        std::cerr << "Error al crear la ventana" << std::endl;
        glfwTerminate();
        return -1;
    }
}
```

```

}

// Asignamos el contexto de OpenGL a la ventana, hace que todas las llamadas de
OpenGL afecten a esa ventana
glfwMakeContextCurrent(window);

// Cargamos las funciones de OpenGL con GLAD
if (!gladLoadGLLoader((GLADloadproc)glfwGetProcAddress)) {
    std::cerr << "Error al inicializar GLAD" << std::endl;
    return -1;
}

// Configuramos el viewport, área de ña venta donde OpenGLdibujará
// El (0,0) son las coordenadas de la esquina inferior izquierda
glViewport(0, 0, SCR_WIDTH, SCR_HEIGHT);

// Bucle de renderizado, comprobamos si se debería cerrar la ventana
while (!glfwWindowShouldClose(window)) {
    // Procesamos entradas por teclado
    processInput(window);

    // Limpiamos la pantalla con un color de fondo
    glClearColor(0.2f, 0.3f, 0.3f, 1.0f);
    glClear(GL_COLOR_BUFFER_BIT);

    // Intercambiamos buffers y procesamos eventos pendientes como el teclado o
del raton
    glfwSwapBuffers(window);
    glfwPollEvents();
}

// Terminamos GLFW
glfwTerminate();
return 0;
}

```

2 Modelado

2.1 Tipos de Shaders

Vertex Shader (VS):

- Posición del vértice del triángulo + otros datos (coordenadas de textura, normal, color, etc).
- Vértice con su posición transformada + (opcionalmente) otros atributos (normal, color, etc.) que se envían al siguiente paso en la pipeline

Fragment Shader (FS):

- Información del fragmento a procesar, que puede incluir datos interpolados desde los vértices (coordenadas de textura, normal, color, etc)
- Calcula el color final del píxel si este se debe representar en pantalla.

Procesos posteriores:

- Z-BUFFER: controla la profundidad de los píxeles para manejar la superposición de objetos en la escena.
- Alpha Test: determina si un objeto debe renderizarse en función de su transparencia
- Blending Test.

2.2 Envío de Vértices

Para generar figuras en OpenGL 3.3 se usan **Vertex Array Objet**, que almacenan toda la información del objeto a dibujar. Los VAOs están formados por **Vertex Buffer Objects** (VBOs), que almacenan información relativa a los vértices, colores, normales, etc.

```
void medianteArray() {

    float vertices[] = { 0.0f, 0.0f, 0.0f, // x
                        .5f, 0.0f, 0.0f,  // x
                        0.0f, .5f, 0.0f   // x, y, z
                        };

    // Se crea el vector de floats que recoge los vértices

    GLuint VAO;
    glGenVertexArrays(1, &VAO);
    glBindVertexArray(VAO);
    // Se genera el VAO
    // Se activa el VAO, de manera que las funciones invocadas
    // posteriormente afectarán a dicho VAO

    GLuint VBO;
    glGenBuffers(1, &VBO);
    glBindBuffer(GL_ARRAY_BUFFER, VBO);
    // Se genera el buffer donde se almacenarán los vértices
    // Se activa el buffer, de manera que las funciones
```

```

// invocadas posteriormente afectarán a dicho buffer

glBufferData(GL_ARRAY_BUFFER, sizeof(vertices), vertices, GL_STATIC_DRAW);
// Se asigna el vector de vértices al buffer y al VAO que están "escuchando"
// y se detalla el modo de dibujo

glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 3 * sizeof(float), (void*)0);
glEnableVertexAttribArray(0);
// Se especifica el número que identifica al atributo "vértices", cuántos
elementos
// del vector compone un vértice, la distancia de salto al próximo y dónde
empieza el
// primer vértice, entre otras cosas
// Se activa el buffer con el atributo "vértices" para su uso

glBindBuffer(GL_ARRAY_BUFFER, 0);
glBindVertexArray(0);
// Se desactiva el buffer (ya no "escucha" modificaciones)
// Se desactiva el VAO (ya no "escucha" modificaciones)
}

```

```

glUseProgram(shaderProgram);
//Se indica cuál es el Program Shader a emplear

glBindVertexArray(VAO);
//El VAO especificado "se pone a la escucha" (se activa)

glDrawArrays(GL_LINES, 0, 6);
//Se dibujan las aristas del objeto especificando desde dónde empezar y el número
de vértices a dibujar del VAO

glBindVertexArray(0);
//Se desactiva el VAO para su uso (ya no "escucha")

```

Si se trabaja con más de un atributo (por ejemplo, se añaden colores):

```

void medianteArray() {

// Se crea el vector de floats que recoge los vértices y los colores
float vertices[] = {
    // Vértices          // Colores

```

```

        0.f, 0.f, 0.f,          1.f, 1.f, 0.f, // Vértice 0: posición, color
amarillo
        .5f, 0.f, 0.f,        0.f, 0.f, 1.f, // Vértice 1: azul
        .5f, .5f, 0.f,        0.f, 1.f, 0.f, // Vértice 2: verde
        0.f, .5f, 0.f,        1.f, 0.f, 0.f, // Vértice 3: rojo
        -.5f, .5f, 0.f,       1.f, 1.f, 1.f, // Vértice 4: blanco
        -.5f, 0.f, 0.f,       0.f, 0.f, 0.f, // Vértice 5: negro
        -.5f, -.5f, 0.f,      0.f, 1.f, 1.f, // Vértice 6: cian
        0.f, -.5f, 0.f,       .5f, 1.f, 0.f  // Vértice 7: mitad de 1,1,1
    };

    GLuint VAO;
    glGenVertexArrays(1, &VAO);
    glBindVertexArray(VAO);

    GLuint VBO;
    glGenBuffers(1, &VBO);
    glBindBuffer(GL_ARRAY_BUFFER, VBO);

    glBufferData(GL_ARRAY_BUFFER, sizeof(vertices), vertices, GL_STATIC_DRAW);

    // Se genera el primer atributo ("vértices"), identificado con 0, indicando
    cuántos elementos
    // del vector compone un vértice, el salto al siguiente vértice y dónde empieza
    el primer vértice.
    // Posteriormente, se activa este atributo para su uso.
    glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 6 * sizeof(float), (void*)0);
    glEnableVertexAttribArray(0);

    // Se genera el primer atributo ("colores"), identificado con 1, indicando
    cuántos elementos
    // del vector compone un color, el salto al siguiente color y dónde empieza el
    primer color.
    // Posteriormente, se activa este atributo para su uso.
    glVertexAttribPointer(1, 3, GL_FLOAT, GL_FALSE, 6 * sizeof(float), (void*)(3 *
    sizeof(float)));
    glEnableVertexAttribArray(1);

    glBindBuffer(GL_ARRAY_BUFFER, 0);
    glBindVertexArray(0);

    glDeleteBuffers(1, &VBO);

```

```

    glUseProgram(shaderProgram);
    glBindVertexArray(VAO);
    glDrawArrays(GL_LINES, 0, 8);
}

```

Para evitar escribir varias veces el mismo vértice, se pueden emplear **vértices indexados**, para lo cual se requiere un vector de índices:

```

void medianteArray() {

    float vertices[] = {
        0.0f, 0.0f, 0.0f, // 0
        .5f, 0.0f, 0.0f,  // x
        0.5f, .5f, 0.0f,  // y
        0.0f, 0.5f, .5f   // z
    };

    unsigned int indices[] = { 0, 1, 0, 2, 0, 3 };
    // Se genera el vector de índices, que recoge el orden en el que se recorren los
    vértices

    GLuint VAO;
    glGenVertexArrays(1, &VAO);
    glBindVertexArray(VAO);

    GLuint VBO, EBO;
    glGenBuffers(1, &VBO);
    glGenBuffers(1, &EBO);
    // Se generan los buffers donde se almacenarán los vértices e índices,
    respectivamente

    // Se activa el buffer de vértices y se especifica el vector al que referencia
    glBindBuffer(GL_ARRAY_BUFFER, VBO);
    glBufferData(GL_ARRAY_BUFFER, sizeof(vertices), vertices, GL_STATIC_DRAW);

    // Se activa el buffer de índices y se especifica el vector al que referencia
    // (ELEMENT_ARRAY_BUFFER indica que se trata de un array de índices)
    glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, EBO);
    glBufferData(GL_ELEMENT_ARRAY_BUFFER, sizeof(indices), indices, GL_STATIC_DRAW);

    glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 3 * sizeof(float), (void*)0);
    glEnableVertexAttribArray(0);
}

```

```

    glBindVertexArray(0);
}

glUseProgram(shaderProgram);
glBindVertexArray(VAO);
glDrawElements(GL_LINES, 6, GL_UNSIGNED_INT, 0);
glBindVertexArray(0);

// La función de dibujo es diferente con indexación

```

2.3 Gestión de Vértices

Los vértices especificados en un VAO se almacenan en la pipeline y posteriormente son gestionados en el VS y FS, que se almacenan como archivos de texto en el directorio de uso. El código de un VS podría ser el siguiente:

```

#version 330 core
layout (location = 0) in vec3 aPos;

void main(){
    gl_Position = vec4(aPos.x, aPos.y, aPos.z, 1.0);
}

```

- `layout (location=0) in vec3 aPos` permite almacenar los elementos del atributo identificado como 0 en VA activo en un vector llamado `aPos` de 3 componentes.
- `gl_Position` es una variable predefinida que podremos modificar para alterar la posición de los vértices. En este caso conservamos la de entrada.

El código de un FS podría ser el siguiente:

```

#version 330 core
void main(){
    gl_FragColor = vec4(1, .5, .2, 1.);
}

```

En caso de que el VAO tenga 2 atributos (vértices y colores) el código de ambos shaders podría ser:

```

// Vertex Shader
#version 330 core
layout (location = 0) in vec3 aPos;
layout (location = 1) in vec3 aColor;

```



```

out vec3 ourColor;

void main() {
    gl_Position = vec4(aPos, 1.0);
    ourColor = aColor;
}

```

Para utilizar los shaders en el programa:

3 Transformaciones

En opengl las transformaciones geométricas se aplican en el VS, multiplicando la matriz de transformación por el vértice y devolviendo la nueva posición. La matriz de transformación se calcula en el `main` usando la biblioteca **GLM** (OpenGL Mathematics).

- `glm::mat4 transform`: declaración de la matriz de transformación
- `glm::mat4 ()`: devuelve la identidad 4x4
- `glm::translate(transform, glm::vec3(T_x, T_y, T_z))`: traslación
- `glm::rotate(transform, 0, glm::Vec3(x, y, z))`: rotación
- `glm::scale(transform, glm::vec4(S_x, S_y, S_z))`: escalado

La matriz se envía al VS usando una variable de tipo `uniform`

Para evitar andar usando el `glm::nombrefunc`, vamos a tirar de `C++` y le chantamos en la primera línea del programa:

```
using namespace glm;
```

4 Visión

En opengl el trabajo sobre la cámara se divide en **las matrices de proyección y de vista**. Para que el VS calcule la posición de los vértices, se **multiplican** la matriz de **proyección, de la vista y la del modelo por los vértices** en coordenadas locales. El orden de la multiplicación es relevante porque el producto no es conmutativo.

$$V_{\text{coordendas clip space}} = M_{\text{proyección}} \times M_{\text{vista}} \times M_{\text{modelo}} \times V_{\text{coordenadas locales}}$$

- `glm::mat4 view = glm::lookAt(glm::vec3(x, y, z), glm::vec3(x, y, z), glm::vec3(x, y, z));` para definir la **matriz de vista**

- `glm::mat4 proj = glm::ortho(left, right, top, down, near, far);`: para realizar una **proyección ortográfica**
- **R** = right
- **L** = left
- **T** = top
- **B** = bottom
- **N** = near
- **F** = far

$$\begin{bmatrix} \frac{2}{R-L} & 0 & 0 & -\frac{(R+L)}{(R-L)} \\ 0 & \frac{2}{T-B} & 0 & -\frac{(T+B)}{(T-B)} \\ 0 & 0 & \frac{2}{F-N} & -\frac{(F+N)}{(F-N)} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- `glm::perspective(fov, aspect, near, far);`: para realizar una proyección **ortográfica**.

$$\begin{bmatrix} \frac{1}{\text{Aspect} \cdot \tan(\text{fovy}/2)} & 0 & 0 & 0 \\ 0 & \frac{1}{\tan(\text{fovy}/2)} & 0 & 0 \\ 0 & 0 & \frac{N+F}{N-F} & \frac{2FN}{N-F} \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

Las matrices se envían al VS usando una variable de tipo **uniform**.

Para que al **reescalar la ventana** se mantengan las **proporciones** de la imagen hay que actualizar el aspecto de la perspectiva al nuevo tamaño de la ventana:

5 Luces

5.1 Modelo de Iluminación Básico

El color de un objeto es fruto de **su color original multiplicado** por el **color de la luz** con la que se ilumina.

5.2 Luz Ambiente

$$I = I_A * K_A$$

5.3 Luz Difusa

Necesitamos pasarle el shader:

- La atenuación
- Las coordenadas de las normales de las caras
- La posición de la luz
- El punto de cálculo para ver la dirección de la luz $\vec{L}$

$$I = I_l \times \cos(\theta) = I_l \times K_D \times (\vec{N} \cdot \vec{L})$$

5.4 Luz Especular

$$I = I_L \times K_S \times \cos^n(\alpha) = I_L \times K_S \times (\vec{R} \cdot \vec{V})^n$$

Hay que pasarle al FS, además de lo ya dicho, la posición de la cámara.